Technical Documentation

Project: HisabDo AI Copilot
Department: AI/ML
Owner: Muhammad Taha
Role: AI/ML Team Lead & AI Architect

---

1. Purpose

This technical documentation defines the proposed technical foundation of **HisabDo AI Copilot**.

Its purpose is to ensure that future developers and teams can understand:

- How the AI system is structured
- How requests flow through the system
- How AI modules communicate
- How verified financial data is accessed
- How AI is integrated with the existing HisabDo application
- What security controls are required
- What limitations currently exist
- What technical requirements are needed for future implementation

This document is designed to support future development, integration, testing, maintenance, and scalability.

---

2. HisabDo AI Copilot Architecture

The proposed system follows a modular architecture:

User

↓

React Frontend

↓

Express Backend

↓

FastAPI AI Service

↓

AI Orchestrator

↓

Knowledge AI / Financial AI / Insight Engine

↓

Verified Data Sources

↓

Response Engine

↓

User Response

The architecture separates the responsibilities of:

- User interface
- Application backend
- AI processing
- Knowledge retrieval
- Financial data access
- Business logic
- Response generation

This separation improves security, maintainability, and scalability.

---

3. Complete AI Request Workflow

Every AI request should follow a controlled workflow:

User Query

↓

Input Processing

↓

Language Detection

↓

Input Normalization

↓

Intent Classification

↓

AI Orchestrator

↓

Request Routing

↓

Relevant AI Module

↓

Verified Context

↓

Response Generation

↓

Response Validation

↓

User Response

The AI system should not send every query directly to an LLM.

Instead, the request is first analyzed and routed to the appropriate module.

---

4. AI Orchestrator

The **AI Orchestrator** is the central routing layer of HisabDo AI Copilot.

Its responsibility is to determine:

What is the user asking?

What data is required?

Which AI module should process the request?

Is verified data required?

What type of response should be returned?

Example:

User:

How much does Ali owe me?

Intent:

CUSTOMER_BALANCE_QUERY

Required Module:

Financial AI

Data Requirement:

Verified Financial Data

Another example:

User:

How do I create a customer?

Intent:

FEATURE_GUIDANCE

Required Module:

Knowledge AI / RAG

Data Requirement:

Verified Product Documentation

---

5. Intent Classification

The intent classification layer identifies the purpose of the user's request.

Initial intent categories may include:

PRODUCT_INFORMATION

FAQ_QUERY

FEATURE_GUIDANCE

CUSTOMER_BALANCE_QUERY

TRANSACTION_QUERY

EXPENSE_QUERY

RECEIVABLE_QUERY

PAYABLE_QUERY

BUSINESS_INSIGHT

ANALYTICS_QUERY

RECOMMENDATION_QUERY

REPORT_HELP

BACKUP_HELP

GENERAL_CONVERSATION

UNKNOWN

Intent classification determines how the AI Orchestrator routes the request.

---

6. Input and Language Processing

Before a request reaches the AI module, the system should process the input.

The processing flow may include:

User Input

↓

Input Validation

↓

Language Detection

↓

Input Normalization

↓

Intent Detection

The initial AI system may support:

- English
- Urdu
- Roman Urdu

Roman Urdu requires additional normalization because users may write the same word differently.

Example:

kitna

kitnay

kitny

udhar

udhaar

udharr

paisa

paisay

pese

The normalization layer should improve intent understanding without changing the user's intended meaning.

---

7. Chatbot Architecture

The chatbot acts as the primary conversational interface.

User

↓

Chat UI

↓

Chat API

↓

AI Orchestrator

↓

Intent Detection

↓

Relevant Module

↓

Response Generation

↓

Chat UI

The chatbot may support:

- Product questions
- FAQs
- Feature guidance
- Help requests
- Financial questions
- Recommendations

The chatbot should not independently access sensitive application data.

---

8. FAQ Assistant Architecture

The FAQ Assistant provides answers based on verified HisabDo information.

User Question

↓

FAQ Search

↓

Relevant FAQ

↓

Verified Content

↓

Response Generation

↓

User Response

The FAQ Assistant should be used for simple and frequently asked questions.

Example categories:

- Getting started

- Customers
- Transactions
- Expenses
- Reports
- Backup
- Offline functionality

---

9. RAG Pipeline

RAG stands for **Retrieval-Augmented Generation**.

The purpose of RAG is to provide the AI with relevant HisabDo information before generating an answer.

HisabDo Documents

↓

Document Cleaning

↓

Text Chunking

↓

Embedding Generation

↓

Vector Database

↓

User Question

↓

Query Embedding

↓

Similarity Search

↓

Relevant Content

↓

LLM

↓

Final Response

RAG should reduce unsupported answers by grounding responses in verified knowledge.

---

10. Knowledge Base Requirements

The AI knowledge base may include:

Knowledge Base

│

├── FAQs

├── Website Pages

├── Feature Descriptions

├── Help Content

├── Product Guides

├── Documentation

├── Blog Articles

├── Support Information

└── User Guides

Before AI ingestion, content should be:

- Accurate
- Current
- Relevant
- Organized
- Cleaned
- Categorized
- Verified

Unverified content should not automatically become a trusted AI knowledge source.

---

11. Website Content Requirements for AI

The AI team should identify all relevant content from the HisabDo website.

Required content may include:

- Product overview
- Feature descriptions
- Customer management information
- Transaction workflows
- Expense management
- Reports
- Backup and restore
- Offline functionality
- Help and support information
- FAQs
- Blog content

Missing or outdated content should be identified before knowledge base integration.

---

12. Frontend Integration Points

The React frontend can provide the following AI integration points:

- AI chat widget
- Help assistant
- FAQ search
- Dashboard insights
- Smart recommendations
- Feature suggestions
- AI notifications
- Feedback controls

Proposed communication:

React Component

↓

Authenticated API Request

↓

Express Backend

↓

AI Service

The frontend should not directly connect to sensitive AI data sources or databases.

---

13. Backend and API Integration

The existing application backend should act as a controlled integration layer.

React

↓

Express API

↓

FastAPI AI Service

The Express backend should handle responsibilities such as:

- Authentication
- Authorization
- Request validation
- Rate limiting
- User context
- API routing
- Logging

The AI service should focus on:

- Intent detection
- AI orchestration
- RAG processing
- Knowledge retrieval
- AI response generation

---

14. React → Express → FastAPI Communication

The recommended request flow is:

User

↓

React Chat UI

↓

POST /api/ai/chat

↓

Express Backend

↓

Authenticated Internal Request

↓

FastAPI AI Service

↓

AI Orchestrator

↓

Relevant AI Module

↓

Structured Response

↓

Express Backend

↓

React UI

This approach prevents the frontend from directly controlling sensitive AI services.

---

15. Verified Financial Data Access Rules

Financial data must be treated differently from general knowledge.

Required flow:

User Question

↓

Financial Intent Detection

↓

Authorization Check

↓

Approved Financial API

↓

Application Database

↓

Business Logic

↓

Verified Result

↓

AI Explanation

↓

User Response

Examples of verified financial requests:

- Customer balance
- Outstanding amount
- Monthly expenses
- Receivables
- Transaction summaries

---

16. No Direct LLM Database Access

A fundamental architecture rule is:

The LLM must never directly access the HisabDo database.

The LLM should only receive controlled and verified information.

Correct architecture:

LLM

↓

Controlled API Request

↓

Backend Service

↓

Business Logic

↓

Database

The database should never be directly exposed to an AI model.

---

17. Authentication and Authorization

AI requests involving user or financial data must require authentication.

The system should:

- Identify the authenticated user
- Validate the session or token
- Pass authorized user context
- Restrict data to the correct user
- Apply permission checks before financial data access

Example:

User A

↓

AI Request

↓

Authorization Check

↓

Only User A's Authorized Data

No AI request should be able to retrieve another user's financial information.

---

18. API Security Requirements

The API layer should include:

- Authentication
- Authorization
- Input validation
- Rate limiting
- Request size limits
- Secure error responses
- Logging
- Request tracking

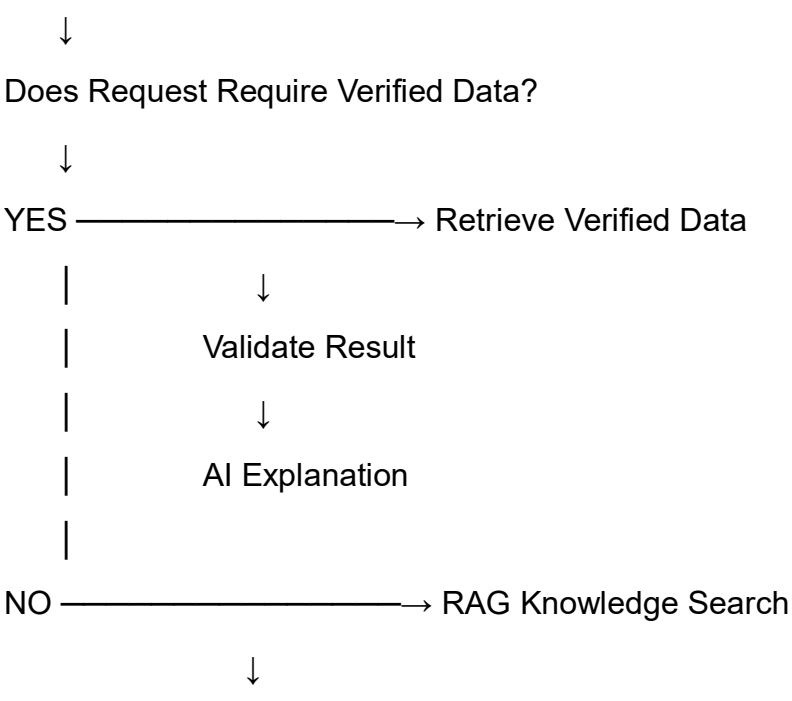
The AI service should also validate:

- Input format
- Request type
- User permissions
- Allowed operations

---

19. Anti-Hallucination Architecture

The AI should not be treated as the source of truth for financial information.



Verified Context

↓

AI Response

The core principle is:

Verified Systems = Facts and Calculations

AI = Understanding and Explanation

---

20. Error Handling and Fallback Behavior

The system should handle errors safely.

Example scenarios:

AI Service Unavailable

Fallback:

Basic Help Message

Knowledge Not Found

Fallback:

Tell the user that verified information was not found.

Financial API Failure

Fallback:

Do not generate or guess financial information.

Unknown Intent

Fallback:

Ask the user for clarification.

The system should prefer a safe fallback over an incorrect answer.

---

21. Testing Requirements

The future AI system should include:

Functional Testing

- Product questions

- FAQs
- Financial queries
- Unknown queries
- Invalid input

AI Testing

- Intent accuracy
- Response relevance
- Unsupported answers
- Hallucination detection

RAG Testing

- Retrieval quality
- Relevant chunk selection
- Knowledge coverage

Multilingual Testing

- English
- Urdu
- Roman Urdu
- Mixed-language input

Integration Testing

- React to backend
- Backend to AI service
- AI to knowledge base
- AI to financial APIs

---

22. Technical Dependencies

Future implementation may depend on:

- React
- Node.js
- Express.js

- Python
- FastAPI
- MongoDB
- Ollama or another AI model provider
- Sentence Transformers
- FAISS or Qdrant

The final technology selection should be based on:

- Cost
- Performance
- Scalability
- Hardware requirements
- Multilingual support
- Integration requirements

---

23. Scalability Requirements

The architecture should support future growth.

Possible scalability requirements include:

- Separate AI service deployment
- Independent AI modules
- Replaceable LLM providers
- Replaceable vector databases
- API versioning
- Background document processing
- Caching
- Monitoring
- Usage analytics

The architecture should allow the system to scale without requiring a complete redesign.

---

24. Current POC Limitations

The current POC and research stage may have limitations such as:

- Limited knowledge base
- Limited test queries
- Incomplete website documentation
- Limited multilingual testing
- Basic intent handling
- No production-scale infrastructure
- Limited financial API integration
- Local hardware limitations

These limitations should be documented clearly rather than hidden.

---

25. Future Technical Requirements

Future implementation may require:

- Production AI service
- Expanded knowledge base
- Improved RAG evaluation
- Full backend integration
- Financial API tools
- Authentication integration
- Authorization controls
- Monitoring
- AI feedback collection
- Performance testing
- Security testing
- Deployment strategy

---

26. Overall Technical Workflow

AI Use Cases

↓

AI Enhancement Proposal

↓

Improved AI POC

↓

Technical Documentation

↓

Implementation Roadmap

↓

Future HisabDo AI Integration

---

27. Final Day 20–25 Deliverables

AI Enhancement Proposal

+

Improved AI POC

+

AI Use Cases

+

Implementation Roadmap

+

Technical Documentation

Conclusion

This technical documentation provides the foundation required for future implementation of **HisabDo AI Copilot**.

The proposed architecture ensures that AI functionality remains modular, secure, and scalable. General questions should use verified knowledge through FAQ and RAG systems, while financial questions must use controlled APIs and verified application data.

The most important technical principle remains:

AI understands, routes, and explains. Verified application systems retrieve data, calculate values, and provide the source of truth.

This documentation, together with the **AI Enhancement Proposal, AI Use Cases, Improved AI POC, and Implementation Roadmap**, provides a complete technical and planning foundation for the next phase of HisabDo AI integration.